RV32I CPU로 INT8 선형계층 빠르게 계산하기

EE362 Project 1 · Release 1.4

박기호 (어셈블리 최적화) · 김대현 (CPU 파이프라인) · 2026-11-17

과제: 곱셈기 없는 CPU로 Y = BIAS + W·X

Y[j] = BIAS[j] + Σ X[i]·W[j][i] (i = 0..7, j = 0..3)

- 입력 X·W 는 INT8, BIAS·Y 는 INT32. 데이터 위치는 case 마다 바뀌어 0x300 설정 블록에서 매번 읽는다
- 명령은 기본 8개(ADD SUB AND ADDI LW SW BEQ JAL)만 사용. MUL 등 M 확장은 금지
- 관문: 공개 16 + 숨김 8 case 정답, 고정 포트, 합성 후 setup/hold 통과. 하나라도 FAIL 이면 PPA 0점
- 점수 = 12.5·T_best/T + 6.25·A_best/A + 6.25·P_best/P → 실행시간 T 가 면적·전력의 2배
- T = 평균 cycle × 주기, A = CPU + IMEM + DMEM 면적, P = CPU + 메모리 전력

결론: EX 분기 5단 파이프라인 + int8_fast, 1.62 ns

31.6×	1.0307 µs	100,343	15.56 mW	16 / 16
기준본 대비 빨라짐	실행시간 T	면적 A (µm²)	전력 P	공개 case PASS

31.6× 는 세 가지 곱이다

14.73×

프로그램

8135.7 → 552.5 cycle

1.05×

단일 사이클 주기

4.00 → 3.82 ns

2.05×

CPU 구조

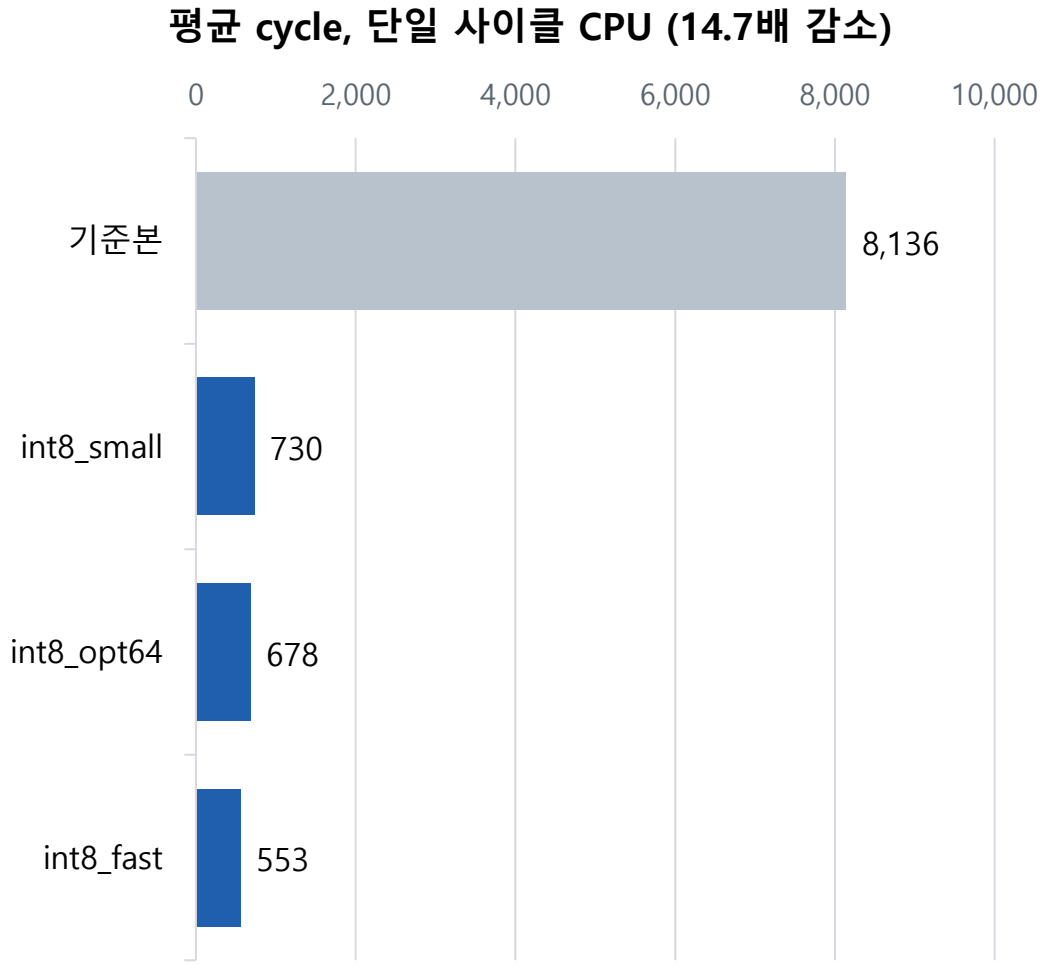
단일 → EX 파이프라인

기준본 T ≈ 32.54 µs (단일 사이클 + 배포 프로그램, 4.00 ns) → 1.0307 µs

① 프로그램: 비트 직렬 곱셈을 네 행이 함께 쓴다

- X[i] 의 비트 k 가 1 이면 네 누산기에 W[j][i]·2^k 을 더한다. 비트 7 은 부호라 뺀다
- 2^k 은 add w,w,w 로 매번 2배, 비트 검사 1번(and+beq)을 출력 4행이 공유
- 기준본의 반복 덧셈 곱 + SCRATCH 저장·재로드를 없앴다
- int8_fast 는 루프를 펼쳐 분기를 줄였다. 대가로 IMEM 이 256 구간

프로그램	word	IMEM	평균 cycle
기준본	57	64	8,135.7
int8_small	48	64	729.5
int8_opt64	58	64	677.9
int8_fast	192	256	552.5



② CPU: 세 가지 구조를 같은 조건에서 비교

	단일 사이클	파이프라인 ID	파이프라인 EX
분기 판정 위치	같은 cycle	ID 단	**EX 단**
taken 분기·JAL 손실	0	1 cycle	**2 cycle**
lw 직후 사용	0	1 cycle stall	**1 cycle stall**
최소 통과 주기	3.82 ns	2.36 ns	**1.62 ns**
int8_fast 평균 cycle	552.5	594.4	**636.3**
int8_fast 실행시간 T	2.1105 μs	1.4027 μs	**1.0307 μs**

세 폴더는 rtl/CORE.v 만 다르다. TOP·CONTROLLER·TB·메모리·SDC·프로그램 HEX 는 동일. 나머지 데이터 의존은 포워딩으로 0 cycle.

③ 주기를 정하는 경로와 넣은 개선

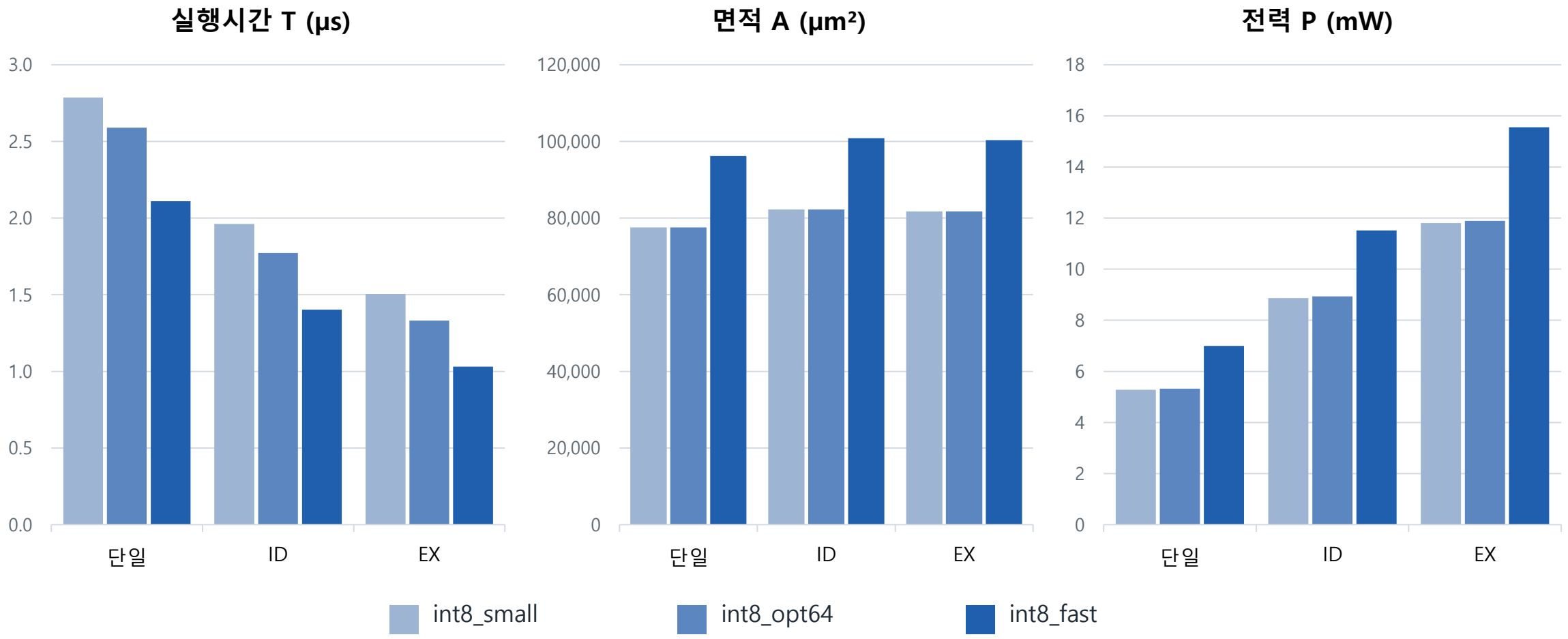
가장 느린 경로 (setup 보고서)

- 단일 3.82 ns: INSTR 입력 → 레지스터 읽기 → ALU → ALURESULT 출력. 입출력 지연 1.00 ns 씩이 한 경로에 같이 들어간다
- ID 2.36 ns: EX 의 ALU 결과 → ID 비교기로 포워딩 → 분기 판정 → PC. 분기 손실 1 cycle 의 대가
- EX 1.62 ns: PCE 레지스터 → 분기 목표 덧셈기 → PC 선택. 남은 건 덧셈기 하나

cycle 수를 바꾸지 않는 개선 4가지

- BEQ 같음 판정을 뺄셈 결과 대신 ALU 와 병렬인 비교기로 (EX)
- 포워딩 select 를 ID 에서 미리 계산해 ID/EX 에 저장
- 버블 때 데이터 레지스터는 두고 제어 비트만 0 → flush 팬아웃 축소
- result mux 를 MEM 단으로 옮겨 MEM/WB 에 32비트만 저장 (FF 66개 감소)

④ PPA: 9개 조합 (공식 flow, 공개 16 case)



T 는 EX 가, A·P 는 단일 사이클이 가장 작다. T 배점이 2배라 EX + int8_fast 가 표 안에서 가장 높다.

⑤ 트레이드오프

- EX 는 단일보다 cycle 이 15.2% 많지만 주기가 2.36배 짧아 T 가 2.05배 짧다
- ID 는 EX 보다 cycle 이 6.6% 적지만 주기가 45.7% 길어 T 가 36.1% 길다
- 면적의 77.8% 가 메모리 → CPU 구조를 바꿔도 A 는 5% 안에서만 움직인다
- 전력은 EX 가 단일의 2.22배. 그러나 한 번 실행 에너지(P×T)는 14.8 / 16.2 / 16.0 nJ 로 비슷하다
- int8_fast 는 opt64 대비 A +22.8%, P +30.9% 이지만 T −22.5%. T 배점이 2배라 fast 를 골랐다

⑥ 기능 증거와 주기 탐색

검증

- 서버 공식 flow: 9개 조합 모두 공개 16/16 PASS, timing PASS
- 로컬 ModelSim: 프로그램 4종 × 16 case + 실행 중 RESET 재인가 + 해저드 시험 + 무작위 프로그램 150개를 ISA 모델과 store 순서·주소·데이터·최종 DMEM 까지 비교 → CPU 마다 219/219 일치
- 계약 검사 check_release.py: TOP 48 포트, 합성 목록, 공통 SDC 통과

주기 탐색 (EX, 0.02 ns 간격)

주기 ns	setup slack ns	통과
1.70	+0.0031	O
1.68	+0.0007	O
1.66	+0.0002	O
1.64	−0.0014	X
1.62	+0.0026	O ← 선택
1.60	−0.0000	X

공식 run_ppa.tcl 과 같은 compile 로 합성, setup·hold 모두 0 이상인 최소 주기. 합성 결과가 주기에 단조롭지 않아(1.64 실패, 1.62 통과) 전부 돌려 본다. hold slack 은 모두 0.103 ns 이상.

⑦ 재현 절차 (수업 서버)

```
# 제출 패키지에서 (R1.4 공통 flow)
python3 tools/check_release.py   # 포트·합성 목록·SDC 계약
bash run_sim.sh                  # VCS 로 공개 16 case 기능 + activity
bash run_ppa.sh                  # DC 합성 · STA · CPU 전력 + SRAM 비용

# 세 CPU × 세 프로그램 비교를 한 번에
cd 1110_compare && bash run_all.sh 2>&1 | tee run_all.log
#   → results/summary.md, <CPU>/comparisons/<프로그램>_P<주기>/ppa_summary.json
```

바꾼 SDC 값은 synth/common.sdc 의 -period 숫자 하나뿐. 합성은 compile (compile_ultra 미사용).

한계와 질문에 대비할 것

- 모든 숫자는 공개 16 case 기준. 공식 성적은 숨김 8개를 더한 24 case 로 나온다
- 타이밍 여유가 EX 2.6 ps · 단일 0.8 ps · ID 0.3 ps. RTL 을 고치면 주기를 다시 찾아야 한다
- 분기 예측기는 없다(항상 PC+4). 기본 8명령만 구현, 선택 명령은 쓰지 않았다
- 전력은 배치·배선 전 추정값 (case 별 RTL SAIF 를 합성 결과에 입혀 cycle 가중 평균)
- 기준본(단일 + 배포 프로그램)의 A·P 는 서버에서 재지 않았다. 기준 T 는 로컬 cycle × 4.00 ns